TinkerWithMe

tinkerwithanne@gmail.com
Nairobi, Kenya

Bluetooth Control Lab · Full Curriculum

2-session series (2 hrs each) · 11–15 yrs · Intermediate → Advanced

Full facilitator curriculum **PREMIUM**

Overview

Control an Arduino gadget wirelessly from a phone over Bluetooth, then build a custom app to command it.

DETAIL	
Track	Arduino Robotics & Electronics
Format	2-session series (2 hrs each)
Ages	11–15 yrs
Level	Intermediate → Advanced
Price	from KES 7,000
Standalone	No
Prerequisites	Button controlled LED (free, recommended)

Course Overview

Bluetooth Control Lab takes students from "it works over a wire" to "it works from across the room" and finally to "I built the app that drives it." Over two 2-hour sessions, students wire up an HC-05 Bluetooth module alongside LEDs and a small DC motor, pair an Android phone with the Arduino, and issue commands from a serial terminal app to understand exactly what is travelling over the wireless serial link. This demystifies Bluetooth — it is simply UART serial communication carried over radio instead of a USB cable.

In the second session, students graduate from a generic terminal app to building their own custom Android app in MIT App Inventor, complete with named buttons, a device picker, and a live-updating display. The Arduino is upgraded to also read an ultrasonic distance sensor and stream data back to the phone, so students

experience genuine two-way wireless communication — commands going out, sensor data coming back — inside an app they designed themselves.

The take-home is twofold: a working Bluetooth-controlled gadget (LEDs + motor on a breadboard, driven by an HC-05) and a fully functional custom Android app the student built and can keep modifying. Both are demoed together in a final showcase where each student pairs their phone, opens their own app, and drives their own gadget live.

Learning Outcomes

- Wire and safely power an HC-05 Bluetooth module, including protecting its 3.3V RX pin from the Arduino's 5V TX output
- Explain how Bluetooth Classic behaves as a transparent serial (UART) link and use SoftwareSerial to keep it separate from the USB programming port
- Write and modify Arduino command-parsing code (switch/case on incoming serial characters) to control multiple outputs
- Drive a DC motor safely from a low-current microcontroller pin using a transistor switch and flyback diode, with a separate power source
- Design a multi-screen-ready Android app in MIT App Inventor using BluetoothClient, ListPicker, Button, Label and Clock components
- Implement two-way communication: sending single-character commands out and parsing streamed sensor text coming back
- Debug a full-stack wireless system (hardware ↔ Bluetooth pairing ↔ app) using a structured checklist rather than guesswork

Materials Master List

ITEM	SPEC	QTY PER PAIR	NOTES
Arduino Uno R3	Uno R3 or compatible clone	1	Nerokas/Pixel Electronics stock these; used both sessions
HC-05 Bluetooth module	6-pin, with onboard voltage regulator, default baud 9600	1	Buy pre-flashed slave-mode modules to save setup time
Breadboard	Half-size, 400 tie-points	1	

Jumper wire pack	Male-male and male-female, assorted lengths	1 pack (~40 wires)	
LEDs	5mm, red / green / yellow	1 each colour	
Resistors	220Ω (x4), 1kΩ (x1), 2kΩ (x1)	1 set	220Ω for LEDs; 1k/2k form the HC-05 RX voltage divider
NPN transistor	2N2222 or BC547	1	Motor driver switch
Flyback diode	1N4007	1	Across motor terminals, protects transistor
DC hobby motor	3–6V small brushed motor	1	
Battery holder + batteries	4xAA holder (6V) or 9V clip	1	Separate power for motor; common ground with Arduino
HC-SR04 ultrasonic sensor	Standard 4-pin (VCC, Trig, Echo, GND)	1	Used in Session 2 for sensor readback
USB cable	USB-A to USB-B	1	For programming; unplug from PC once running on battery/BT
Android smartphone	Bluetooth Classic capable, Android 6.0+	1 (student-owned)	Must allow app install from unknown sources for exported APK if needed
Serial Bluetooth Terminal app	Free Android app (Play Store)	1 install	Install and test-pair before Session 1 if possible
MIT App Inventor account	Browser-based, ai2.appinventor.mit.edu, Google login	1 per pair	Create accounts in advance to save time
Laptop/PC with Arduino IDE	Arduino IDE 2.x installed, drivers pre-tested	1 per pair	
Small screwdriver / multimeter	Shared facilitator toolkit	1 per table	For continuity checks during debugging

Session Plans

Session 1: Connect & Command

Objectives

- Wire an HC-05 module safely and pair it with a phone

- Send single-character commands from a terminal app to switch LEDs and control a motor
- Understand that Bluetooth here is just serial data carried wirelessly

Timing

- 0:00–0:10 — Welcome, recap serial communication basics, introduce today's goal
- 0:10–0:25 — Explain HC-05 pinout, voltage divider requirement, and why we use SoftwareSerial
- 0:25–0:55 — Guided build: LEDs + resistors on breadboard, then HC-05 wiring with divider
- 0:55–1:10 — Upload sketch via USB, test with Serial Monitor before introducing Bluetooth
- 1:10–1:20 — Break
- 1:20–1:40 — Pair phone with HC-05 (code 1234 or 0000), open Serial Bluetooth Terminal, test LED commands
- 1:40–1:55 — Add motor circuit (transistor + diode + external battery), test forward/stop commands
- 1:55–2:00 — Wrap-up: what worked, what commands do what, preview Session 2

Step-by-step

1. Recap: ask "what is Bluetooth actually sending?" Establish it's just characters over a serial line, same as the USB Serial Monitor.
2. Draw the HC-05 pinout on the board: VCC, GND, TXD, RXD, STATE, EN. Explain HC-05 logic is 3.3V — its TXD is safe to feed directly into Arduino RX, but its RXD pin must never see 5V, hence the voltage divider from Arduino TX.
3. Explain why we use SoftwareSerial on pins 10/11 instead of the hardware Serial pins 0/1 — those are needed for USB programming and Serial Monitor debugging.
4. Guide LED wiring first (simpler, testable immediately): each LED anode through a 220Ω resistor to its digital pin, cathode to GND rail.
5. Guide HC-05 wiring: VCC to 5V, GND to GND rail, TXD to Arduino pin 10, RXD to a junction of 1kΩ (to Arduino pin 11) and 2kΩ (to GND) — this divider drops the 5V TX signal to roughly 3.3V.

6. Upload the sketch over USB. Open Serial Monitor at 9600 baud, type '1' and Enter — confirm red LED turns on. This proves the code logic works before adding wireless complexity.
7. Introduce pairing: on the phone, go to Bluetooth settings, scan, find "HC-05", pair using code 1234 (or 0000 if that fails).
8. Open Serial Bluetooth Terminal app, connect to the paired HC-05, send the same single characters ('1','0','2','3', etc.) and confirm LEDs respond exactly as they did over USB.
9. Add the motor: transistor base to Arduino pin 9 via a 220Ω resistor (already have; reuse or add one more), collector to motor negative lead, emitter to GND, motor positive to external battery positive, battery negative and Arduino GND joined (common ground is essential). Diode across motor leads, band toward the positive/battery side.
10. Test 'F' and 'S' commands from the terminal app. Discuss why the motor needs its own battery — the Arduino's 5V pin cannot supply enough current for a motor.
11. Circulate and debug pairs individually; use the common mistakes table below as a triage checklist.

Build detail

Materials: Arduino Uno, breadboard, HC-05 module, 3 LEDs (red/green/yellow), 4×220Ω resistors, 1×1kΩ resistor, 1×2kΩ resistor, NPN transistor, 1N4007 diode, small DC motor, 4xAA battery holder, jumper wires, USB cable.

Wiring (component → Arduino pin):

- Red LED anode (via 220Ω) → pin 5; cathode → GND
- Green LED anode (via 220Ω) → pin 6; cathode → GND
- Yellow LED anode (via 220Ω) → pin 7; cathode → GND
- HC-05 VCC → Arduino 5V
- HC-05 GND → Arduino GND
- HC-05 TXD → Arduino pin 10 (SoftwareSerial RX)
- HC-05 RXD → junction of 1kΩ resistor (other end to Arduino pin 11) and 2kΩ resistor (other end to GND) — this is the voltage divider
- Transistor base → 220Ω resistor → Arduino pin 9
- Transistor emitter → GND rail (shared with Arduino GND and battery negative)

- Transistor collector → motor negative lead
- Motor positive lead → battery pack positive
- Battery pack negative → GND rail (common ground with Arduino)
- 1N4007 diode across motor leads, banded end toward motor positive/battery side

```
#include <SoftwareSerial.h>

// HC-05 TXD -> pin 10 (Arduino RX), HC-05 RXD -> pin 11 (Arduino TX, via divider)
SoftwareSerial btSerial(10, 11);

const int redPin    = 5;
const int greenPin   = 6;
const int yellowPin  = 7;
const int motorPin   = 9;

void setup() {
  pinMode(redPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
  pinMode(yellowPin, OUTPUT);
  pinMode(motorPin, OUTPUT);

  Serial.begin(9600);     // USB Serial Monitor, for debugging
  btSerial.begin(9600);   // HC-05 default baud rate

  Serial.println("Bluetooth Control Lab - Session 1 ready");
  Serial.println("Commands: 1/0=Red on/off 2/3=Green 4/5=Yellow F=Motor forward S=Motor stop");
}

void loop() {
  // Listen for commands arriving wirelessly from the phone
  if (btSerial.available()) {
    char cmd = btSerial.read();
    handleCommand(cmd);
  }

  // Also listen on USB Serial Monitor so we can test without the phone
  if (Serial.available()) {
    char cmd = Serial.read();
    handleCommand(cmd);
  }
}

void handleCommand(char cmd) {
  switch (cmd) {
    case '1': digitalWrite(redPin, HIGH);    btSerial.println("RED ON");    break;
    case '0': digitalWrite(redPin, LOW);     btSerial.println("RED OFF");   break;
    case '2': digitalWrite(greenPin, HIGH);  btSerial.println("GREEN ON");  break;
    case '3': digitalWrite(greenPin, LOW);   btSerial.println("GREEN OFF"); break;
    case '4': digitalWrite(yellowPin, HIGH); btSerial.println("YELLOW ON"); break;
    case '5': digitalWrite(yellowPin, LOW);  btSerial.println("YELLOW OFF");break;
    case 'F': analogWrite(motorPin, 200);    btSerial.println("MOTOR FORWARD"); break;
    case 'S': analogWrite(motorPin, 0);      btSerial.println("MOTOR STOP");    break;
    default:
      // Ignore newline characters, spaces, or unrecognised commands
      break;
```

```
  }
}
```

Checks for understanding

- Why can't we connect HC-05's RXD directly to the Arduino's TX pin?
- If you send 'x' from the terminal app and nothing happens, is that a bug? Why or why not?
- Why does the motor need its own battery instead of running off the Arduino's 5V pin?

Common mistakes & fixes

MISTAKE	FIX
HC-05 won't pair / not visible	Check VCC/GND wiring and that module's LED is blinking fast (pairing mode); confirm power via multimeter
LEDs don't respond over Bluetooth but work over USB Serial Monitor	Confirm phone is connected to the correct paired device in the terminal app, and baud rate is set to 9600
Motor doesn't spin	Check common ground between battery and Arduino; verify transistor orientation (emitter/base/collector) and diode direction
Random LED flickering with no commands sent	Voltage divider on HC-05 RXD is wired wrong or missing — check resistor values and junction point
Uploading code fails while HC-05 connected	HC-05 on pins 10/11 shouldn't interfere, but disconnect BT power briefly if upload errors occur, or remove TXD/RXD wires during upload

Session 2: Build Your App

Objectives

- Design a custom Android app in MIT App Inventor with buttons that send Bluetooth commands
- Add an ultrasonic distance sensor to the Arduino and stream live readings back to the phone
- Achieve full two-way communication: control the gadget and read live data in one custom app

Timing

- 0:00–0:10 — Recap Session 1, introduce today's plan: build our own app
- 0:10–0:20 — Tour of App Inventor interface: Designer view vs Blocks view

- 0:20–0:50 — Guided build: BluetoothClient, ListPicker (device connect), control buttons
- 0:50–1:00 — Test app: pair, connect, press buttons, confirm LEDs/motor respond
- 1:00–1:10 — Break
- 1:10–1:25 — Wire the HC-SR04 ultrasonic sensor onto the breadboard alongside existing circuit
- 1:25–1:40 — Upload updated Arduino sketch, test distance readings via Serial Monitor
- 1:40–1:55 — Add Label + Clock in App Inventor to receive and display live distance readings
- 1:55–2:00 — Final test end-to-end, save/export project (.aia), showcase prep

Step-by-step

1. Open ai2.appinventor.mit.edu, start a new project named "BTGadgetControl".
2. In Designer view, drag in a **ListPicker** (rename to `ListPicker1`, text "Connect to HC-05"), a **BluetoothClient** (non-visible component, from Connectivity palette), six **Buttons** (rename btnRedOn, btnRedOff, btnGreenOn, btnGreenOff, btnYellowOn, btnYellowOff), two more Buttons (btnMotorForward, btnMotorStop), a **Label** (lblStatus) to show connection state, another Label (lblDistance) to show sensor readings, and a **Clock** component (Clock1, TimerInterval 500, TimerEnabled unchecked initially).
3. Switch to Blocks view. Build: `ListPicker1.BeforePicking` → set `ListPicker1.Elements` to `BluetoothClient1.AddressesAndNames` (this populates the picker with paired devices).
4. Build: `ListPicker1.AfterPicking` → call `BluetoothClient1.Connect` with address `ListPicker1.Selection`; if it returns true, set `lblStatus.Text` to "Connected" and set `Clock1.TimerEnabled` to true; if false, set `lblStatus.Text` to "Connection failed".
5. For each control button's `.Click` event, call `BluetoothClient1.SendText` with the matching single character in quotes: btnRedOn → "1", btnRedOff → "0", btnGreenOn → "2", btnGreenOff → "3", btnYellowOn → "4", btnYellowOff → "5", btnMotorForward → "F", btnMotorStop → "S". These must match the Arduino's switch/case exactly.
6. Build `Clock1.Timer` event: if `BluetoothClient1.IsConnected` and `BluetoothClient1.BytesToReceive > 0`, call `BluetoothClient1.ReceiveText` with numberOfBytes -1 (reads all available), store the result in a local/global variable "incoming", then if `incoming` contains the text "DIST:", use the text-splitting blocks to strip the prefix and set `lblDistance.Text` to the remaining number plus " cm".

7. Test the app using the built-in "Test" live preview via the MIT AI2 Companion app if phones support it, or export directly as an APK for install — either works, but installing the APK is the authentic "took it home" experience.
8. Now update the hardware: add the HC-SR04 sensor to the same breadboard, wiring Trig and Echo to two free digital pins.
9. Upload the Session 2 Arduino sketch (below), which keeps all Session 1 commands and adds a 'D' command plus automatic distance broadcasting every 500ms.
10. Reconnect the app, confirm buttons still control LEDs/motor, and confirm the distance label updates live as students wave a hand in front of the sensor.
11. Discuss: this is genuine two-way communication — commands flow phone→Arduino, sensor data flows Arduino→phone, over the same Bluetooth serial link.

Build detail

Materials (adds to Session 1 build): HC-SR04 ultrasonic sensor, 2 extra jumper wires.

Wiring (component → Arduino pin), in addition to Session 1 wiring:

- HC-SR04 VCC → Arduino 5V
- HC-SR04 GND → Arduino GND
- HC-SR04 Trig → Arduino pin 12
- HC-SR04 Echo → Arduino pin 13

```
#include <SoftwareSerial.h>

// HC-05 TXD -> pin 10 (Arduino RX), HC-05 RXD -> pin 11 (Arduino TX, via divider)
SoftwareSerial btSerial(10, 11);

const int redPin    = 5;
const int greenPin   = 6;
const int yellowPin  = 7;
const int motorPin   = 9;
const int trigPin    = 12;
const int echoPin    = 13;

unsigned long lastSend = 0;
const unsigned long sendInterval = 500; // send a distance reading every 500ms

void setup() {
  pinMode(redPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
  pinMode(yellowPin, OUTPUT);
  pinMode(motorPin, OUTPUT);
```

```
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);

  Serial.begin(9600);
  btSerial.begin(9600);

  Serial.println("Session 2 ready - two-way Bluetooth link");
  Serial.println("Commands: 1/0 2/3 4/5 = LEDs, F/S = Motor, D = request distance now");
}

// Sends a Trig pulse and measures the Echo pulse width to calculate distance in cm
long readDistanceCM() {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  // Timeout after 30ms (roughly 5 metres) to avoid an infinite wait if no echo returns
  long duration = pulseIn(echoPin, HIGH, 30000);
  long distance = duration * 0.034 / 2; // speed of sound conversion, divided by 2 for round trip

  if (duration == 0) {
    distance = -1; // no echo detected, out of range or no object
  }
  return distance;
}

void handleCommand(char cmd) {
  switch (cmd) {
    case '1': digitalWrite(redPin, HIGH);    break;
    case '0': digitalWrite(redPin, LOW);     break;
    case '2': digitalWrite(greenPin, HIGH);  break;
    case '3': digitalWrite(greenPin, LOW);   break;
    case '4': digitalWrite(yellowPin, HIGH); break;
    case '5': digitalWrite(yellowPin, LOW);  break;
    case 'F': analogWrite(motorPin, 200);    break;
    case 'S': analogWrite(motorPin, 0);      break;
    case 'D': {
      long d = readDistanceCM();
      btSerial.print("DIST:");
      btSerial.println(d);
      break;
    }
    default:
      break; // ignore anything else (newlines, spaces, unknown chars)
  }
}

void loop() {
  // Incoming commands from the phone app
  if (btSerial.available()) {
    char cmd = btSerial.read();
    handleCommand(cmd);
  }

  // Automatically broadcast a fresh distance reading every 500ms
  // so the app's Label updates live without needing to ask each time
  if (millis() - lastSend >= sendInterval) {
    lastSend = millis();
```

```
      long d = readDistanceCM();
      btSerial.print("DIST:");
      btSerial.println(d);
    }
  }
```

Checks for understanding

- In the app, which block tells the phone to actually connect to a specific Bluetooth device?
- Why does the Arduino sketch send text starting with "DIST:" instead of just the number?
- What would happen if two students' phones both tried to connect to the same HC-05 at once?

Common mistakes & fixes

MISTAKE	FIX
ListPicker shows no devices	Phone must already be paired with HC-05 in Android Bluetooth settings before opening the app
App connects but buttons do nothing	Check SendText blocks use the exact same characters as the Arduino's switch/case (case-sensitive)
Distance label never updates	Confirm Clock1.TimerEnabled was set true after a successful connection, and check the "DIST:" split logic in Blocks
Distance reads -1 constantly	Check Trig/Echo wiring and that nothing is blocking or too close (<2cm) to the sensor
App and Serial Monitor can't both be connected to HC-05	Bluetooth Classic (SPP) allows only one active connection at a time — disconnect one before using the other

Assessment & Showcase

Assess students continuously across both sessions using a simple three-point rubric per milestone: *wiring correctness* (does the circuit match the diagram, is the voltage divider present, is the ground common), *code understanding* (can the student explain what each line of the switch/case does and modify a command), and *debugging independence* (can they use the common-mistakes table to self-diagnose before asking for help). Facilitators should circulate every 15–20 minutes during build blocks and log a quick pass/needs-support mark per pair rather than waiting until the end.

The final showcase happens in the last 10–15 minutes of Session 2. Each pair pairs their phone live, opens their own custom app (not the generic terminal), presses their buttons to demonstrate LED and motor control, and shows the live distance label updating as they wave an object near the sensor. "Done" means: the app connects without facilitator help, all six LED commands and both motor commands work, and the distance label updates automatically at least once every second without needing a button press. Students who reach this bar have achieved full two-way wireless control and should export their .aia project file and APK to keep.

Extension & Home Challenges

- **Easy:** Add a third button state — instead of separate on/off buttons, add a single "Toggle Red LED" button in the app that alternates state each press (hint: track state with a global variable in Blocks, and add a matching 'T' command in the Arduino sketch).
- **Medium:** Replace the fixed motor speed with a Slider component in App Inventor that sends a speed value (e.g. "S150" for PWM 150) instead of a fixed 'F' command, and update the Arduino code to parse a number after a letter prefix using Serial string parsing instead of single characters.
- **Hard:** Add a safety/alert feature — modify the Arduino sketch so that if the ultrasonic sensor detects an object closer than 10cm, it automatically sends a distinct message (e.g. "ALERT") over Bluetooth regardless of the polling interval, and modify the App Inventor blocks so that receiving "ALERT" changes the background colour of the screen and plays a sound using the Sound component.